

Coding Challenge

Task Description: You are working as a **DevOps Engineer** in an agile software team. In the last sprint, the team had the task of developing a robust, platform-independent C# application to manage orders in an SMT (Surface-Mount Technology) manufacturing environment. In the current sprint the application should be **verified, validated and released** to the customer. You have picked this task. At the end of the sprint, you will present your results to the stakeholders to facilitate an active discussion about the implementation and quality. Providing an outline of the quality aspects, CI/CD and test automation approach is required. **Using AI** in every step is allowed and **expected** if beneficial. Please document how AI was used and how its output was validated.

Existing Application provided to you (VS Code):

The application supports **CRUD operations for Orders, Boards, and Components**. The relationships between the entities should be as follows:

- An Order can include one or more Boards, and a Board can be produced in one or multiple orders.
- A Board can contain one or more Components, and a Component can be placed on one or multiple boards.
- The following attributes should be included for each entity:
- **Order: Name, Description, Order Date**
- **Board: Name, Description, Length, Width**
- **Component: Name, Description, Quantity**

The team has extended these entities with additional properties as needed to accurately represent the relationships between the objects (e.g., foreign keys, IDs, reference lists).

Supported actions:

- The application allows the creation, editing, search, and removal of one or more: components, boards and orders.
- The application allows a download of an order – simulate a download to a production line where the order will be produced.

The application has a Web front end and serializes data to JSON for interoperability and persists it using SQLite.

The solution includes a logging mechanism using a commonly adopted framework, to capture relevant actions and errors.

Object-oriented programming principles (e.g., SOLID, DRY), standard design patterns, and unit tests for some representative methods to ensure code quality is used.

Focus DevOps & Test Automation Topics:

The following technical requirements and activities are focused on the current position and shall be implemented to demonstrate advanced knowledge, completeness, and best practices:

- **Verification and validation of the Application:** Check with (automated) tests if the application fulfills the requirements correctly and provides the end-user functionality as described. In case of missing functionalities, defects, used antipatterns, missing logs etc. provide a fix proposal to the team (e.g.: before fix test is “red”, after fix test is “green”). Refactor as needed. If the fix is too complex it is OK to only outline the solution without implementation.
- **Implement feedback from Sprint Review Meeting:** 1. Board cannot be deleted if there is an active order for it. 2. The performance should be optimized for a board with 50.000+ components. If the fix is too complex it is OK to only outline the solution without implementation.
- **Version Control:** Use a free version control system such as GitHub or Azure DevOps to host the project. All code changes should be committed to the remote repository. Upon completion, make the repository accessible for review (public or restricted access).
- **CI/CD Pipeline:** Set up a build and release pipeline using a free service like GitHub Actions or Azure DevOps Pipelines. The pipeline should: Automatically build the application on each commit and run automated tests. Generate a SBOM.

- **Dockerization / Containerization:** Containerize the entire solution (API, database, frontend) using Docker. Provide a Dockerfile and optionally a docker-compose.yml file to simplify deployment and local development. The containerized app should be runnable both locally and in the cloud. Use Infrastructure as Code (IaC) if possible.
- **[Optional] Deploy** the application to a free-tier cloud provider (e.g., Microsoft Azure). Explain how the deployed application can be **monitored**. Use Infrastructure as Code (IaC) if possible.

Please outline in the presentation (~ 30 minutes + questions) where and how AI was used or might be used for every mentioned activity. Invested effort should not exceed 4-6 hours.